Accepted Manuscript

Route Relaxations on GPU for Vehicle Routing Problems

Marco Antonio Boschetti, Vittorio Maniezzo,
Francesco Strappaveccia

PII: S0377-2217(16)30802-5
DOI: 10.1016/j.ejor.2016.09.050
Reference: EOR 14011

To appear in: *European Journal of Operational Research*

Received date: 4 October 2015
Revised date: 12 September 2016
Accepted date: 27 September 2016

Please cite this article as: Marco Antonio Boschetti, Vittorio Maniezzo, Francesco Strappaveccia, Route Relaxations on GPU for Vehicle Routing Problems, *European Journal of Operational Research* (2016), doi: 10.1016/j.ejor.2016.09.050

This is a PDF file of an unedited manuscript that has been accepted for publication. As a service to our customers we are providing this early version of the manuscript. The manuscript will undergo copyediting, typesetting, and review of the resulting proof before it is published in its final form. Please note that during the production process errors may be discovered which could affect the content, and all legal disclaimers that apply to the journal pertain.

ACCEPTED MANUSCRIPT

Highlights

- A description of route relaxations proposed in the literature for vehicle routing problems, more detailed than the currently published ones.
- A study on the implementation of the route relaxations on GPU and of the benefits that can thereby be achieved.
- Two new asymmetric VRP instances of large size corresponding to real-world problems.

ACCEPTED MANUSCRIPT

ACCEPTED MANUSCRIPT

Route Relaxations on GPU for Vehicle Routing Problems

Marco Antonio Boschetti

University of Bologna, Department of Mathematics, via Sacchi 3, Cesena, Italy

Vittorio Maniezzo

University of Bologna, DISI, via Sacchi 3, Cesena, Italy

Francesco Strappaveccia*

University of Bologna, DIN, via Sacchi 3, Cesena, Italy

Abstract

State-Space Relaxation (SSR) is an approach often used to compute by dynamic programming (DP) effective bounds for many combinatorial optimization problems.

Currently, the most effective exact approaches for solving many Vehicle Routing Problems (VRPs) are DP algorithms making use of SSR for computing their bounding components. In particular, most of these make use of the q-route and ng-route relaxations. The bounding procedures based on these relaxations provide good quality bounds but they are often time consuming to compute, even for moderate size instances.

In this paper we investigate the use of GPU computing for solving q-route and ng-route relaxations. The results prove that the proposed GPU implementations are able to achieve remarkable computing time reductions, up to 40 times with respect to the sequential versions.

Keywords: Dynamic Programming, Route Relaxation, Parallel Algorithms, GPU Computing, CUDA

*Corresponding author

ACCEPTED MANUSCRIPT

ACCEPTED MANUSCRIPT

1. Introduction

The Vehicle Routing Problem (VRP) is among the most studied problems in combinatorial optimization, and retains unabated interest both because, though simple to state, it enjoys intriguing mathematical properties and because it can be quickly specified into problems of primary economic interest. The literature on the core problem variants and on the possible real-world variations got huge after the seminal paper which introduced it ([1]), and includes dedicated books ([2], [3]) and, more recently, also dedicated working groups of research associations ([4]).

The core problem can be quickly stated as finding a least cost set of routes to service a number of customers from a central depot, given a cost matrix specifying the travel cost between all customer/depot locations. The problem can be further complicated, by adding real world constraints. A commonly included constraint assumes that the routes are to be travelled by vehicles in order to deliver or to collect goods from customers, thus the total amount of goods loaded on each truck cannot exceed its capacity, in weight or in volume. This gives rise to the Capacitated VRP variant (CVRP). Alternatively, each customer can ask either for a delivery or for a collection of goods (or both), yielding the Pickup and Delivery variants (VRPPD), including the case where there are coupled pairs of pickups and deliveries. In case all deliveries of each route are to be made first, then all collections, we have the CVRP with backhauls (VRPB). A further quite common constraint considers feasible time windows for the visits at the customers (VRPTW). Moreover, in small area settings each truck could go back to the depot to reload (multi-trip VRP, MTVRP), while in bigger areas the use of more depots is common by the vehicles of the fleet (multi-depot VRP, MDVRP). The vehicles of the fleet can be equal or different (heterogeneous fleet CVRP, HVRP), they could be requested to return to the depot they started from, or not (open CVRP, OVRP), they could be requested to repeat the same routes with a given periodicity over the planning horizon (periodic VRP, PVRP), etc.

ACCEPTED MANUSCRIPT

Furthermore, all listed constraints, and many more coming from operational practice, can be freely combined to model actual use cases. For example, a recent work on city logistics operational optimization ([5]) models its problem as a CVRP with time windows, multi-trip, heterogeneous fleet and pickup and delivery.

Given its theoretical and practical relevance, the VRP witnessed a wealth of diverse approaches for its solution and still fosters a lively research community studying either exact or heuristic methods or a combination of them, as recently done by matheuristic approaches ([6]). A detailed survey is clearly out of scope for this paper, in the following we will recall just a few contributions.

Exact approaches are of more direct interest for this paper. Again, different approaches have been used, ranging from dynamic programming ([7]) to branch and bound ([8]), from branch and cut ([9]) to column generation ([10]). In all cases, a central feature is the ability to compute tight lower bounds. Again, different approaches for computing bounds are proposed in the literature and, recently, bounding procedures based on non-elementary paths, such as q-routes ([8]) and most notably ng-routes ([11, 12]) appear to be particularly effective for the CVRP.

GPU (Graphics Processing Unit) computing is achieving increasing interest among the optimization community, given the possibility to significantly speed up tasks at the core of many approaches of interest, thus to ultimately achieve substantial efficiency improvements ([13]). The number of contributions in the field is in fact increasing at a fast rate, with actual applications especially in the area of bioinformatics.

One optimization approach particularly prone to be implemented using GPU computing is dynamic programming, because both its basic data structures (essentially a n-dimensional array) and its staged computations can take advantage of the GPU architecture. However, dynamic programming is not the only approach which can benefit from GPU computing, in fact [14] in their extensive survey, report results obtained also by different metaheuristics, ACO, PSO, genetic algorithms, differential evolution, and devote a section to local search

ACCEPTED MANUSCRIPT

and related hybrids. Notwithstanding, the majority of the applications so far, are based on dynamic programming, and are usually in the area of bioinformatics. An interesting survey of GPU-accelerated bioinformatic applications can be found in [15], where several widely different areas of application are overviewed, usually reporting implementations which achieve speedups of 20x or more, with some case where two orders of magnitude speedups are possible.

Applications to combinatorial optimization problems have already been reported for a number of problems, mainly for the core version of basic problems. Thus, [16] proved that on a consumer GPU it is possible to obtain a speed-up of 20 on the Knapsack Problem and [17] and [18] provided an extensive spectrum of graph problems (Depth First Search, etc.) which are substantially accelerated by using a GPU. [19] and [20] used GPU acceleration on algorithms like the Bellman-Ford and Dijkstra for solving the Single Source Shortest Path Problem (SSSPP). The GPU has given remarkable results also on the All Pairs Shortest Path Problem solved with the Floyd-Warshall algorithm ([21], [22]). Actual industrial applications of manycore accelerated codes are still scarce, one can be found for the Two-Dimensional Guillotine Cutting Problem ([23]).

In this paper we propose the first work where GPU computing is used for implementing vehicle routing optimization components. Specifically we propose a GPU implementation of the dynamic programming procedures for computing q-route and ng-route bounds. The implementation on GPU of an optimization algorithm is a complex task that involves the study of tailored data structures and corresponding routines. The paper reports in detail the choices we made to achieve the most efficient parallel implementation of the q-route and ng-route bound computation routines and substantiates this with computational results on standard problem benchmarks from the literature.

The paper is structured as follows. In Section 2 we describe the CVRP and we report the set partitioning model used in the literature for it. In Section 3 bounding procedures based on q-route and ng-route relaxations are described, whereas their GPU implementation is described in Section 4. Computational results are reported and discussed in Section 5 and conclusions are drawn in

ACCEPTED MANUSCRIPT

Section 6.

2. Problem Description and Mathematical Formulations

The CVRP consists in finding the least-cost set of routes to be serviced by a homogeneous fleet of m vehicles, each with capacity Q, in order to service each of n customers, whose index set is V_1. All routes start and return to a common depot, conventionally indexed by 0. Let $V = V_1 \cup \{0\}$. Input data consist of the requests q_i of each customer $i = 1, \ldots, n$, and of the travelling costs c_{ij}, $i = 0, \ldots, n$, $j = 0, \ldots, n$, between each pair of customers and between each customer and the depot.

The problem can thus be defined on a complete weighted graph $G = (V, A, C)$, where $A = \{(i, j) : i, j \in V\}$, and $C = \{c_{ij} : i, j \in V\}$ is the corresponding, possibly asymmetric, cost matrix. In real-world applications, G is typically an overlay graph superimposed on an actual road network, and vertices in V correspond to geocoded facilities, while arcs in A correspond to least-cost paths, computed according to the metric to minimize (e.g., distance, time, etc.).

The problem can be formulated in different ways, we refer the reader to [2] for a thorough overview. In this paper we just consider the set partitioning model, which is the most common one making use of q-route and ng-route relaxations.

The set partitioning formulation, originally proposed by [24], associates a decision variable x_ℓ to each feasible vehicle route, that is, to each route that can be travelled by a vehicle, leaving the depot, servicing a subset of customers that collectively do not exceed the vehicle capacity and finally returning to the depot.

Let $\mathscr{R}$ be the index set of all feasible routes, let c_ℓ be the cost of route $\ell \in \mathscr{R}$, and let $a_{i\ell}$ be a binary coefficient, which is equal to 1 if and only if customer

ACCEPTED MANUSCRIPT

$i \in V_1$ belongs to route $\ell \in \mathscr{R}$. The formulation is as follows.

$$z = \min \sum_{\ell \in \mathscr{R}} c_\ell x_\ell \tag{1}$$

$$s.t. \sum_{\ell \in \mathscr{R}} a_{i\ell} x_\ell = 1, \qquad i \in V_1 \tag{2}$$

$$\sum_{\ell \in \mathscr{R}} x_\ell = m \tag{3}$$

$$x_\ell \in \{0, 1\}, \qquad \ell \in \mathscr{R} \tag{4}$$

Constraints (2) ensure that each customer is serviced by exactly one feasible route and constraint (3) impose the fleet cardinality. It is noteworthy that, in case the cost matrix satisfies the triangle inequality, equalities (2) could be turned into *greater than or equal to* inequalities, thus turning the problem into an extended set covering problem, which is computationally easier to deal with.

3. Relaxations based on Dynamic Programming

Among the exact methods for solving variants of VRP (e.g., CVRP) the most effective ones are often based on a set partitioning model and a column generation approach. These methods start with a limited set of columns and they iteratively add new columns, which have the *potential* to enter the optimal solution. The new columns are *generated* by solving the so-called *pricing problem*.

The pricing problem corresponds to the Elementary Shortest Path Problem with Resource Constraints (ESPPRC), also known as Resource Constrained Elementary Shortest Path Problem (RCESPP), which is strongly NP-Hard as shown by [25]. This problem is defined on a set $\mathbf{R} = (R^1, \ldots, R^m)$ of *available resources* and on a graph $G(V, A, C')$, where V is the set of vertices containing n customers and two vertices s and t representing the source and the sink of the path, which represent the depot of the corresponding VRP, A is the set of arcs, and C' is the cost matrix. Each arc $(i, j) \in A$ has an associated cost c'_{ij}, possibly negative in case of pricing problems because it corresponds to the cost

ACCEPTED MANUSCRIPT

c_{ij} *penalized* by dual variables, and a resource vector $\mathbf{r}_{ij} = (r_{ij}^1, \ldots, r_{ij}^m)$, which specifies the amount of each resource that is needed to make use of that arc. For any path $P = ((s = i_0, i_1), \ldots, (i_{h-1}, i_h = t))$, the cost is given by the sum of the costs of the arcs belonging to P, i.e., $c(P) = \sum_{j=1}^{h} c'_{i_{j-1},i_j}$. The path P is feasible if the resource consumption of the path does not exceed the available resources, i.e. $\sum_{j=1}^{h} r^k_{i_{j-1},i_j} \leq R^k$, for every resource $k = 1, \ldots, m$. The problem consists in finding the least cost feasible elementary path from the vertex s to vertex t.

The ESPPRC can be solved with Dynamic Programming (DP) as, for example, proposed by [26], [27]. The state-space size of the exact DPs proposed in the literature so far increases too quickly when the size of instances increases. Even for moderate size instances the state-space size could be too large for solving the problem in realistic time on a powerful workstation. Therefore, [28] proposed the State-Space Relaxation (SSR), where the state-space of the DP is relaxed to compute lower (resp. upper) bounds to the original minimization (resp. maximization) problem. The main idea behind SSR is well summarized by Righini and Salani [29], who observe that with SSR the search space explored by DP is projected onto a lower dimensional space so that only the minimum cost state is retained among all the corresponding states in the higher dimensional space. In the literature some interesting state-space relaxations for the ESPPRC are proposed by [8], [27],[29],[30], and [11]. These algorithms, as all dynamic programming algorithms, trade space for time, enumerating all the interesting solutions for the relaxed problem. In these cases, the elementarity of the path is relaxed or partially relaxed to find interesting paths, where cycling is not avoided but it is reduced by limiting the amount of available resources (e.g., weight, time) or by applying specific rules (e.g., avoid cycles of cardinality two or involving a given subset of vertices). These *almost* elementary paths, can be the base for the construction of feasible solutions or they can be a good starting point for the computation of tight bounds. In the next sections we describe three of these methods (q-route, through-q-route and ng-route) and we analyze the characteristics of each of them.

ACCEPTED MANUSCRIPT

3.1. q-Route Relaxation

Let $f(q,i)$ be the cost of the least cost path $P = (0, i_1, \ldots, i_k = i)$ (not necessarily elementary) from the depot to customer i with total load $q = q(P) = \sum_{h=1}^{k} q_{i_h}$. Such a path is called *q-path*. A q-path with the additional arc $(i,0)$ is called *q-route* and has cost $f(q,i) + d_{i0}$.

Christofides et al. [8] extend the q-path definition to impose that each path should not contain loops of two arcs. Let $\pi(q,i)$ be the vertex just prior to i in the path corresponding to $f(q,i)$. Let $\phi(q,i)$ be the cost of the least cost path ending at vertex i with the constraint that the vertex $\gamma(q,i)$ preceding i is not equal to $\pi(q,i)$. The cost $h(q,j,i)$ of the least cost path of load q from the depot 0 to i, with j just prior to i and without loops of two arcs is evaluated as follows:

$$h(q,j,i) = \begin{cases} f(q-q_i,j) + d_{ji}, & \text{if } \pi(q-q_i,j) \neq i \\ \phi(q-q_i,j) + d_{ji}, & \text{otherwise} \end{cases} \tag{5}$$

Given the function $h(q,j,i)$, the functions $f(q,i)$ and $\phi(q,i)$ can be computed as follows:

$$\begin{aligned} f(q,i) &= \min_{j \neq i} \{h(q,j,i)\} \\ \pi(q,i) &= \operatorname{argmin}_{j \neq i} \{h(q,j,i)\} \end{aligned} \tag{6}$$

and

$$\phi(q,i) = \min_{j \neq \pi(q,i), j \neq i} \{h(q,j,i)\} \tag{7}$$

The recursion is initialized by setting $f(q,i) = d_{0i}$ and $\pi(q,i) = 0$ for $q = q_i$, $f(q,i) = \infty$ and $\pi(q,i) = -1$ for every $q \neq q_i$, and $\phi(q,i) = \infty$ for every q.

The pseudocode of the algorithm is as follows:

ALGORITHM Q-PATHS$(n, Q, \mathbf{q}, \mathbf{d})$

```
1  // Initialization
2  for q = 0, ..., Q do
3      for i = 1, ..., n do
4          φ(q,i) = ∞
5          if q = q_i
```

ACCEPTED MANUSCRIPT

then $f(q,i) = d(0,i)$, $\pi(q,i) = 0$

else $f(q,i) = \infty$, $\pi(q,i) = -1$

// Recursion

for $q = 0, \ldots, Q$ **do**

  for $i = 1, \ldots, n$ **do**

    for $j = 1, \ldots, n$ **do**

      if $(i \neq j) \wedge (\pi(q - q_i, j) \neq i)$

        then $h(q,j,i) = f(q - q_i, j) + d(j,i)$

        else $h(q,j,i) = \phi(q - q_i, j) + d(j,i)$

    $f(q,i) = \min_{j \neq i} \{h(q,j,i)\}$

    $\pi(q,i) = \text{argmin}_{j \neq i} \{h(q,j,i)\}$

    $\phi(q,i) = \min_{j \neq \pi(q,i), j \neq i} \{h(q,j,i)\}$

return $\mathbf{f}, \boldsymbol{\pi}, \boldsymbol{\phi}$.

Using this dynamic programming procedure we can find *forward* shortest paths from the depot to each vertex i using a vehicle of capacity Q avoiding loops involving two arcs.

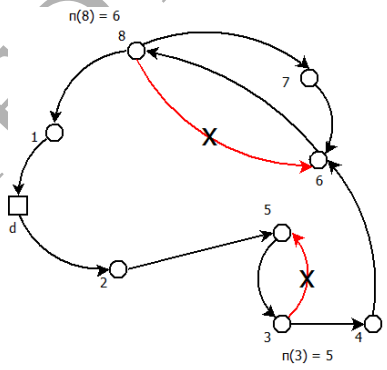

Figure 1: q-path avoiding 2 vertices loop

Considering the example in Figure 1, we set $\pi(q,8) = 6$ for the state (q,i) that is generated for vertex $i = 8$ from a state of vertex $j = 6$. Hence, it will not be possible to generate a state $(q',6)$ from state $(q,8)$, avoiding loops involving

ACCEPTED MANUSCRIPT

two arcs. However, we cannot yet guarantee the path elementarity.

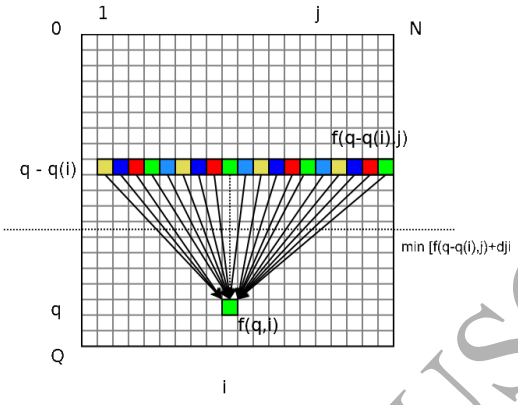

Figure 2: f(q,i) q-path computation

In some situations, as in Section 3.2, we could need to compute also the *backward* q-paths, which are the shortest paths from every vertex i to the depot using a vehicle of capacity Q. When the graph is symmetric (i.e., $d_{ij} = d_{ji}$, for every $i, j \in V$), the values $f^{-1}(q, i)$ of backward q-paths are equivalent to the values of the forward q-paths (i.e., $f(q, i) = f^{-1}(q, i)$). On the other hand, in the case of asymmetric graphs, where $d_{ij} \neq d_{ji}$ for some $i, j \in V$, the backward q-paths can be computed by the same recursion as in the forward case, but applied to the transpose $\mathbf{D}^T$ of the cost matrix $\mathbf{D} = (d_{ij})$.

The q-route relaxation can be computed in pseudo-polynomial time with a complexity of $O(n^2Q)$. In Figure 2 we present graphically the computation of a single $f(q, i)$ value. Notice that for evaluating each state value $f(q, i)$, given $q - q_i$, we have to iterate over all the n vertices, which are saved in contiguous locations in the computer memory, allowing a better computational performance.

3.2. Through-q-Route Relaxation

The through-q-route relaxation (see [8]) is a better relaxation obtained evaluating the cost $\psi(q, i)$ of the least cost route, without loops of two arcs, starting

ACCEPTED MANUSCRIPT

from the depot, passing through i and going back to the depot, with a total load q. The $\psi(q,i)$ values are computed by *joining* two q-paths by using values $f(q,i)$ and $\phi(q,i)$ as follows:

$$\psi(q,i) = \min_{q_i \le \bar{q} \le (q+q_i)/2} \begin{cases} f(\bar{q},i) + f(q+q_i-\bar{q},i), & \text{if } \pi(\bar{q},i) \neq \pi(q+q_i-\bar{q},i) \\ \min\{f(\bar{q},i) + \phi(q+q_i-\bar{q},i), \phi(\bar{q},i) + f(q+q_i-\bar{q},i)\}, & \text{otherwise} \end{cases} \tag{8}$$

The algorithm to compute through-q-routes is as follows:

ALGORITHM THROUGH-Q-ROUTES($n, Q, \mathbf{q}, \mathbf{f}, \boldsymbol{\phi}, \boldsymbol{\pi}$)

```
1  // Recursion
2  for i = 1,...,n do
3      for q = 0,...,Q do
4          ψ(q,i) = ∞
5          for q_i ≤ q̄ ≤ (q+q_i)/2 do
6              q' = q + q_i − q̄
7              if π(q̄,i) ≠ π(q',i)
8                  then if f(q̄,i) + f(q',i) < ψ(q,i)
9                           then ψ(q,i) = f(q̄,i) + f(q',i)
10                 else if f(q̄,i) + φ(q',i) < ψ(q,i)
11                          then ψ(q,i) = f(q̄,i) + φ(q',i)
12                 if φ(q̄,i) + f(q',i) < ψ(q,i)
13                     then ψ(q,i) = φ(q̄,i) + f(q',i)
14 return ψ
```

More informally, this recurrence selects, among all the paths for a given q, the best combination of paths that starts and ends at the depot. Once q-route relaxation has been computed, the through-q-route function $\psi(q,i)$ can be computed in pseudo-polynomial time with a complexity of $O(n^2Q^2)$.

3.3. ng-Route Relaxation

[11] proposed the ng-route relaxation, which is usually more effective than

ACCEPTED MANUSCRIPT

the q-route relaxation, even if there is no dominance relation between them.

The main drawback of q-path is the possibility to often generate paths containing loops of three or more vertices. Procedures to avoid these larger loops are computationally expensive and can address loops of three or four vertices only ([31]). The ng-route relaxation partially solves this problem introducing a supplementary information for the dynamic programming algorithm, which allows the recurrence to *remember* some of the vertices already visited to avoid loops involving them.

Let the neighbor set of vertex i, $N_i \subseteq V$ be a subset of vertices selected according to some criterion, for example the nearest ones to i, such that $i \in N_i$ and $|N_i| \leq \Delta_i$, where Δ_i is the maximum cardinality of the selected neighbors of i. Given a path $P = (0, i_1, \ldots, i_k)$ and the set $V(P)$ of its visited vertices, the subset $\Pi(P)$ of prohibited extensions for the path P is defined as follows:

$$\Pi(P) = \left\{ i_r \in V(P) : i_r \in \bigcap_{s=r+1}^{k} N_{i_s}, r = 1, \ldots, k-1 \right\} \bigcup \{i_k\}. \tag{9}$$

An ng-path $P = (0, i_1, \ldots, i_{k-1}, i_k = i)$ is the least cost path, non-necessarily elementary, starting from the depot, visiting a subset of customers, possibly more than once, ending at vertex i with a total load of $q = \sum_{h=1}^{k} q_i$, such that $i \notin \Pi(P')$, where $P' = (0, i_1, \ldots, i_{k-1})$. The ng-path P is represented by the state (q, i, NG), $NG = \Pi(P)$, and its cost is $f(q, i, NG)$. Functions $f(q, i, NG)$ can be computed by the following recursion:

$$f(q, i, NG) = \min_{(q', j, NG') \in \Psi(q, i, NG)} \{ f(q', j, NG') + d_{ji} \}. \tag{10}$$

where

$$\Psi(q, i, NG) = \left\{ (q - q_i, j, NG') : j \in \Gamma_i^{-1}, NG' \subseteq N_j, j \in NG', NG' \cap N_i = NG \setminus \{i\} \right\} \tag{11}$$

We denote with $L(q, i)$ the set of states (q, i, NG) generated at a given stage (q, i). Notice that $L(q, i)$ is dynamically updated during the computation. The recursion is initialized by setting $f(q_i, i, \{i\}) = d_{0i}$, $L(q_i, i) = \{(q_i, i, \{i\})\}$, and $L(q, i) = \emptyset$, for every $q \neq q_i$. In order to simplify the presentation we assume that $f(q, i, NG) = \infty$, if $(q, i, NG) \notin L(q, i)$.

ACCEPTED MANUSCRIPT

ALGORITHM NG-PATHS$(n, Q, \mathbf{d}, \mathbf{q}, \mathbf{N})$

```
1  // Initialization
2  for q = 0, ..., Q do
3      for i = 1, ..., n do
4          if q_i = q
5              then f(q_i, i, {i}) = d_{0i}
6                   L(q_i, i) = {(q_i, i, {i})}
7              else  L(q, i) = ∅
8  // Recursion
9  for q = 0, ..., Q do
10     for i = 1, ..., n do
11         for j = 1, ..., n do
12             for (q − q_i, j, NG') ∈ L(q − q_i, j) do
13                 if i ∉ NG'
14                     then NG = N_i ∩ NG' ∪ {i}
15                          if f(q − q_i, j, NG') + d_{ji} ≤ f(q, i, NG)
16                              then f(q, i, NG) = f(q − q_i(i), j, NG') + d_{ji}
17                                   L(q, i) = L(q, i) ∪ NG
18 return f
```

In the implementation we replaced $L(q, i)$ with a list of indices to retrieve in O(1) the set NG and its corresponding value $f(q, i, NG)$. Moreover, to define the path $P = \{0, i_1, \ldots, i_k\}$ corresponding to each value $f(q, i, NG)$ we added the index of the state of the previous stage that generated the value. We have also applied the dominance proposed by [11], which states that, given two paths P_1 and P_2, P_1 dominates P_2 if the following three conditions hold: *(i)* $q(P_1) \leq q(P_2)$; *(ii)* $c(P_1) \leq c(P_2)$; and $\Pi(P_1) \subseteq \Pi(P_2)$.

An ng-path with the additional arc $(i, 0)$ is called *ng-route* and its cost is $f(q, i, NG) + d_{i0}$.

ACCEPTED MANUSCRIPT

4. Relaxations on GPU

In this section we give a brief introduction on GPU computing and we describe how to parallelize on GPU the q-route and ng-route relaxations.

The *GPUs* integrate a massive quantity of simple, lightweight cores on the same chip. This makes this solution very attractive for fine-grained, massively parallel tasks. In 2007, NVIDIA introduced *CUDA* (Compute Unified Device Architecture) ([32]), a parallel, general purpose programming model designed to exploit the great computational resources available on a GPU. CUDA is implemented as an extension of the C/C++ programming language.

CUDA programming implements a higher abstraction model than a straight transposition of the hardware architecture of the GPU. The model works at the top level on a *kernel*, which is the portion of code executed asynchronously on the GPU, the remaining of the user code being directly executed on the CPU. The kernel is executed in parallel on a user-defined number of *threads*. The threads are grouped into *blocks*, which are equal cardinality subsets of threads. The blocks are in turns logically arranged into a *grid*, which is a 1-, 2- or 3-dimensional array of blocks. Every block is logically executed on a different *Streaming Multiprocessor* (SM) of the GPU (i.e., *Stream Processor* or *Cluster Processor*), which consists of a set of simple arithmetic cores called *CUDA Cores* (the NVIDIA Fermi architecture has 32 *CUDA Cores* for each SM, Kepler 192).

Every block can accept, in the latest Fermi and Kepler architectures, up to 1024 threads ([32]), even though it will actually simultaneously execute sets of only 32 threads at a time, called *Warps*.

There is a memory hierarchy on these devices, aimed to minimize the communications between the host memory (i.e., the RAM of the PC and also called *Global Memory*) and the device memory (i.e., the memory integrated on the board of the graphic card), via the PCI-Express bus. In fact, besides the core registers, the GPU has an on-board, low-latency memory module accessible to all threads of each single block. The access to Global Memory is a significant bottleneck for performance enhancement in the design of a GPU-accelerated ap-

ACCEPTED MANUSCRIPT

plication. The Global Memory is the only memory visible by the entire grid, and sometimes it is needed for synchronization and data sharing among the blocks, even though its access latency is very high (400-800 memory cycles). There are many techniques used to hide this latency, but they are mainly application-specific, thus often impossible to apply if the peculiar characteristics of the considered algorithm do not allow them. It is therefore crucial to properly manage the access to Global Memory. The effectiveness of this management is measured by the achieved *memory bandwidth*, which quantifies the amount of relevant data which the application can get from the global memory per second.

Dynamic programming algorithms are particularly suitable for parallelization on a GPU architecture where every thread executes a relatively simple computational kernel. We use the CUDA parallel programming model because in our opinion it is the best trade off among portability, usability, and reliability. NVIDIA provides also a good environment for debugging and very effective profiling tools (i.e., Nsight debugger for Microsoft Visual Studio, etc.).

4.1. GPU q-Route

The q-route relaxation is based on equations (5)–(7) which describe the expansion rule to evaluate the value $f(q,i)$ of state (q,i). These equations represent a *forward* recursion which generates a new state from those generated at the previous stages. In particular, all states (q,i), $i=1,\ldots,n$, of a stage q can be evaluated independently using values $f(q-q_i,j)$, $j=1,\ldots,n$, already computed at the previous stages $q-q_i$. Therefore, we can evaluate in parallel every values $f(q,i)$ of the states of stage q, as depicted in Figure 3.

According to the CUDA programming model, we can assign a threads block to each state (q,i) and T threads to each block. We chose an assignment of the workload for computing in parallel not only the values of the states by the recursion (intra-grid parallelism), but also the *minimization* operation required to compute it. In fact, we use the threads of each block to evaluate in parallel the minimization (which is a *parallel reduction*) of equations (6) and (7) (intra-block parallelism) using the method suggested in [33]. Actually, in our implementation

ACCEPTED MANUSCRIPT

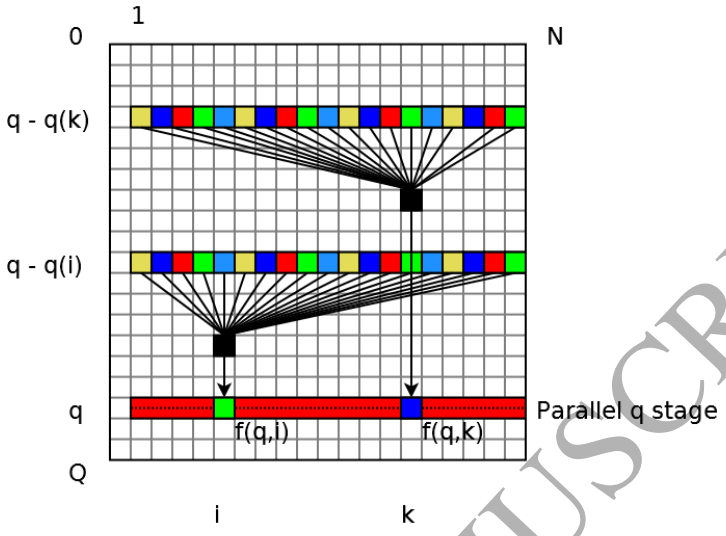

Figure 3: Stage q parallel computation

all data structures have linear access to each element, which means that all matrices are stored in a row-major order, i.e., in our implementation consecutive elements of each row of the matrix are contiguous in memory. However, for the sake of ease, in the description of the algorithm we use two indexes for the matrices.

In the following we provide the pseudo-code for the parallel algorithm and the computation kernel for the q-path algorithm.

ALGORITHM GPU-Q-PATHS($n, Q, \mathbf{q}, \mathbf{d}$)

1 Variables $\mathbf{f}$ and $\boldsymbol{\pi}$ are initialized as in the serial algorithm
2 // Kernel Setup
3 BLOCKS $B = n - 1$, THREADS T, SHARED-MEM $Sh[2 * (n - 1)]$
4 // Main Loop
5 **for** $q' = 0, \ldots, Q$ **do**
6 Q-PATHS-KERNEL<<< B, T, Sh >>>(Q, q', $\mathbf{q}$, $\mathbf{d}$, $\mathbf{f}$, ϕ, $\boldsymbol{\pi}$)
7 **return** $\mathbf{f}, \boldsymbol{\pi}, \phi$

ACCEPTED MANUSCRIPT

The main procedure GPU-Q-Paths starts by defining the dimension of the shared memory for each block of the grid. The dimension is defined so to store the predecessors Γ_i^{-1} of a given vertex i and its corresponding values of $\mathbf{h}$ defined in the equation (5). In our case, since we have supposed G to be complete, the number of predecessors is equal to $n-1$.

Lines 5–6 of the procedure GPU-Q-Paths represent the main loop of the algorithm, where for each stage q of the dynamic programming recurrence the computation kernel for the GPU is iteratively called.

Q-Paths-Kernel$(Q, n, q', \mathbf{q}, \mathbf{d}, \mathbf{f}, \boldsymbol{\phi}, \boldsymbol{\pi})$

1 In the Shared Memory Sh are allocated $\mathbf{h}_{sh}[n-1]$, $\boldsymbol{\pi}_{sh}[n-1]$
2 $thdidx = threadIdx.x$, $Nthds = blockDim.x$, $i = blockIdx.x$
3 $slack = n\%Nthds$, $times = n/Nthds$
4 **if** $thdidx \leq slack$
5 **then** $times = times + 1$
6 // Shared Memory Initialization
7 **for** $t = 0, \ldots, times$ **do**
8 $h_{sh}(thdidx + t * Nthds) = \infty$
9 $\pi_{sh}(thdidx + t * Nthds) = -1$
10 syncthreads()
11 // Partial Minima Computation
12 **for** $t = 0, \ldots, times$ **do**
13 $j = thdidx + t * Nthds$
14 **if** $(i \neq j) \wedge (\pi(q' - q_i, j) \neq i)$
15 **then** $h_{sh}(j) = f(q' - q_i, j) + d(j, i)$
16 **else** $h_{sh}(j) = \phi(q' - q_i, j) + d(j, i)$
17 $\pi_{sh}(j) = j$
18 syncthreads()
19 // First reduction to get $f(q', i)$ and $\pi(q', i)$ values
20 GPU-Reduction($thdidx$, $Nthds$, $times$, $\mathbf{h}_{sh}$, $\boldsymbol{\pi}_{sh}$)
21 **if** $thdidx = 0$

ACCEPTED MANUSCRIPT

22 **then** $f(q',i) = h_{sh}(0)$, $\pi(q',i) = \pi_{sh}(0)$
23 // Second reduction to get $\phi(q',i)$ value
24 $h_{sh}(0) = \infty$, $\pi_{sh}(0) = -1$
25 GPU-REDUCTION($thdidx$, $Nthds$, $times$, $\mathbf{h}_{sh}$, $\boldsymbol{\pi}_{sh}$)
26 **if** $thdidx = 0$
27 **then** $\phi(q',i) = h_{sh}(0)$

The procedure Q-PATHS-KERNEL evaluates $f(q,i)$ for a vertex i. It initializes the shared memory at lines 7–9 and then, at lines 12–18, it computes values $h(q,j,i)$ according to equation (5), storing them in $\mathbf{h}_{sh}$ and the corresponding predecessor j in $\boldsymbol{\pi}_{sh}$. At lines 20–27 it computes by procedure GPU-REDUCTION the minimum values required for evaluating $f(q,i)$ and $\phi(q,i)$. The first call of procedure GPU-REDUCTION computes the value $f(q,i)$, then its value is eliminated from the shared memory, by setting it to infinity, and the second call computes $\phi(q,i)$.

Procedure GPU-REDUCTION is based on the algorithm described in [33] with few modifications to retrieve data in the shared memory and to swap data instead of overwriting them. In lines 2–12 GPU-REDUCTION performs the first stage of the algorithm selecting the most promising values and at lines 14–25 performs the second stage finding the minimum value.

The *syncthreads()* instruction is used to synchronize the threads of each block to perform correctly the subsequent reduction, for finding the minimum. In fact, without that synchronization step the algorithm incurs race conditions. The reduction computes all partial results from each predecessor. Each thread takes care of a subset of nodes and stores in the shared memory the partial results of the expansion of these nodes. After the synchronization, all data in the shared memory are used for the reduction.

GPU-REDUCTION($thdidx, Nthds, times, \mathbf{h}_{sh}, \boldsymbol{\pi}_{sh}$)
1 // Keeping only the most promising value for each thread
2 **for** $t = 0, \ldots times$ **do**

ACCEPTED MANUSCRIPT

```
3     if h_sh(thdidx) > h_sh(thdidx + t * Nthds)
4       then // Swap values
5             f_swap = h_sh(thdidx)
6             h_sh(thdidx) = h_sh(thdidx + t * Nthds)
7             h_sh(thdidx + t * Nthds) = f_swap
8             // Swap Predecessors
9             π_swap = π_sh(thdidx)
10            π_sh(thdidx) = π_sh(thdidx + t * Nthds)
11            π_sh(thdidx + t * Nthds) = π_swap
12  syncthreads()
13  // Reduction
14  for s = 2 * k, k = Nthds/4, ..., 0 do
15    if thdidx < s
16      then if h_sh(thdidx + s) < h_sh(thdidx)
17             then // Swap values
18                  f_swap = h_sh(thdidx)
19                  h_sh(thdidx) = h_sh(thdidx + s)
20                  h_sh(thdidx + s) = f_swap
21                  // Swap Predecessor
22                  π_swap = π_sh(thdidx)
23                  π_sh(thdidx) = π_sh(thdidx + s)
24                  π_sh(thdidx + s) = π_swap
25    syncthreads()
```

4.2. GPU through-q-Route

The through-q-route relaxation, described in Section 3.2, makes use of values $f(q,i)$ and $f^{-1}(q,i)$, which were obtained applying the q-paths relaxation. In this section, for the sake of ease, we only consider the symmetric case.

In order to minimize the transaction between the central memory and the GPU memory, we compute the q-path and, keeping the results on the GPU

ACCEPTED MANUSCRIPT

memory, we perform the GPU algorithm for the through-q-routes.

The computation of each value $\psi(q, i)$, according to expression (8), is independent from the other values, therefore we decided to compute in parallel all values $\psi(q, i)$ for each vertex i of the graph, because capacity Q is usually bigger than n so this is computationally more convenient.

In the following we describe the algorithm which computes the through-q-route relaxation on a GPU.

ALGORITHM GPU-THROUGH-Q-ROUTES($n, Q, \mathbf{f}, \mathbf{q}, \boldsymbol{\phi}, \boldsymbol{\pi}$)

1 // Kernel Setup
2 BLOCKS $B = Q$, THREADS T, SHARED-MEM $Sh[T]$
3 // Main Loop
4 **for** $i = 1, \ldots, n$ **do**
5 THROUGH-Q-ROUTES-KERNEL<<< B, T, Sh >>>(Q, i, $\mathbf{q}$, $\mathbf{d}$, $\mathbf{f}$, $\boldsymbol{\phi}$, $\boldsymbol{\pi}$, $\boldsymbol{\psi}$)
6 **return** $\boldsymbol{\psi}$

The main procedure GPU-THROUGH-Q-ROUTES starts by defining the dimension of the shared memory for each block of the grid. In this case we need only a vector where to store the values $\psi(q, i)$ for each thread.

THROUGH-Q-ROUTES-KERNEL($Q, N, i, \mathbf{q}, \mathbf{f}, \boldsymbol{\phi}, \boldsymbol{\pi}, \boldsymbol{\psi}$)

1 In the Shared Memory Sh is allocated $\mathbf{h}[T]$
2 $thdidx = threadIdx.x$, $NThds = blockDim.x$, $q = blockIdx.x$
3 $\Delta q = (q + q_i)/2 - q_i$, $times = \Delta q / NThds$, $slack = \Delta q \% NThds$
4 **if** $thdidx < slack$
5 **then** $times = times + 1$
6 // Shared Memory Initialization
7 $h(thdidx) = \infty$
8 syncthreads()
9 // Compute the partial values
10 **for** $t = 0, \ldots, times$ **do**

ACCEPTED MANUSCRIPT

```
11      q̄ = thdidx + q_i + (t * NThds)
12      if π(q̄, i) ≠ π(q + q_i − q̄, i)
13        then if h(thdidx) > f(q̄, i) + f(q + q_i − q̄, i)
14                then h(thdidx) = f(q̄, i) + f(q + q_i − q̄, i)
15        else  if h(thdidx) > f(q̄, i) + φ(q + q_i − q̄, i)
16                then h(thdidx) = f(q̄, i) + φ(q + q_i − q̄, i)
17              if h(thdidx) > φ(q̄, i) + f(q + q_i − q̄, i)
18                then h(thdidx) = φ(q̄, i) + f(q + q_i − q̄, i)
19  syncthreads()
20  // Compute the minimum by a parallel reduction
21  for s = 2 * k, k = Nthds/4, ..., 0 do
22      if thdidx < s
23        then if h(thdidx + s) < h(thdidx)
24                then h(thdidx) = h(thdidx + s)
25      syncthreads()
26  // Data Update
27  if thdidx = 0
28     then ψ(q, i) = h(0)
```

Procedure GPU-Through-Q-Routes initializes at lines 1–8 the local variables for the assigned block q. At lines 10–19 the $NThds$ available threads compute the Δq partial values, according to equation (8), and store them in the shares memory for computing the minimum at lines 21–28 by means of the algorithm described in [33].

4.3. GPU ng-Route

The ng-route relaxation is computationally more expensive than the q-route and the through-q-route relaxations and the main problem to face for its implementation is the efficient management of the *ng-sets* generated by expression (9).

The ng-sets associated to each stage (q, i) are dynamically generated, and

ACCEPTED MANUSCRIPT

have a variable size. Dynamic data structures on a GPU environment are not desirable, because their management could be a performance killer. In this section we describe our strategies for addressing this problem and we present the resulting parallel algorithm for the ng-route relaxation on GPU.

Notice from the expression (9) that all the NG sets generated for a given stage (q, i) are completely contained in set N_i and also contain the vertex i. Therefore, the complete enumeration for all the possible NG sets at stage (q, i) is given by all the subsets of N_i containing the vertex i, which are at most $2^{\Delta(N_i)-1}$ (remember that $\Delta(N_i)$ is the maximum size of N_i). For reasonable $\Delta(N_i)$ values, we may easily enumerate a priori all the possible sets NG in a static data structure.

Figure 4 shows how all the states associated to sets NG generated by recursion (9) may be stored in a three-dimensional data structure, where the sizes correspond to the possible loads q, the vertex indexes i, and the indexes of sets NG. For doing that we need to define the *mapping* between indexes and sets NG and to reformulate recursion (10) in its *forward* form:

$$f(q+q_k, k, NG) = \min_{(q,i,NG')\in\Psi'(q+q_k,k,NG)} \{f(q, i, NG') + d_{ik}\}, \forall (q+q_k, k, NG) \in \Phi \tag{12}$$

where

$$\Psi'(q', k, NG) = \{(q, i, NG') : (q, i, NG') \in \Psi(q', k, NG)\} \tag{13}$$

Using expression (12) we can compute independently all the values $f(q + q_k, k, NG)$ from every state (q, i, NG') in the set of stages having load q (see Figure 4).

4.3.1. Transition Mapping

To easily retrieve the index of each set NG, we decided to pre-compute each transition from a set NG associated to vertex j to a set NG' associated to vertex i. Given the starting a vertex j and the index of the set NG associated to it, the *transition map* is able to provide the index of the resulting set NG' associated with the ending vertex i.

ACCEPTED MANUSCRIPT

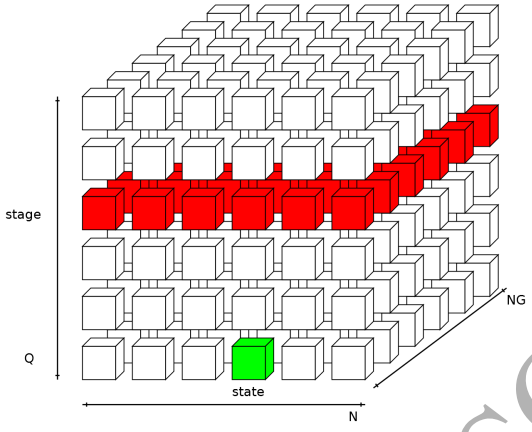

Figure 4: NG path 3-D state space

Let S_i be the sequence obtained from N_i as follows: $S_i = (i_0, i_1, \ldots, i_h)$, where $h = |N_i| - 1$, $i_k \in N_i$ for every $k = 0, \ldots, h$, and $i_k < i_{k+1}$ for every $k = 0, \ldots, h-1$. For each vertex i all the possible sets NG are subset of N_i. Therefore, given a vertex i and a set $NG \subseteq N_i$, we may define a *bit mask* with respect to the sequence S_i as follows: $Mask_i(NG) = (b_0, b_1, \ldots, b_h)$, where $b_k = 1$ if $i_k \in NG$ and $b_k = 0$ otherwise.

Based on this bit mask, we may define a unique *index* for each $NG \subseteq N_i$:

$$Index_i(NG) = \sum_{k=0}^{|S_i|-1} b_k * 2^k \tag{14}$$

Example: Given a vertex $i = 7$ and the sequence $S_i = \{7, 2, 4, 6\} \subseteq N_7$, where $N_7 = \{2, 4, 6, 7, 8, 10\}$ and $\Delta(N_7) = 6$. We have $Mask_i(NG) = \{1, 1, 1, 1, 0, 0\}$ and $Index_i(NG) = 1*2^0 + 1*2^1 + 1*2^2 + 1*2^3 + 0*2^4 + 0*2^5 = 1 + 2 + 4 + 8 + 0 + 0 = 15$.

The index $Index_i(NG)$ gives the possibility to retrieve the set NG on the fly. Moreover, we may define in advance which will be the index for the new set NG' created by expanding another state. Given the starting vertex j, the $Index_j(NG)$ of set NG, and the destination vertex i, the mapping function $Map(j, i, Index_j(NG))$ returns the index $Index_i(NG')$ of the set NG'.

ACCEPTED MANUSCRIPT

4.3.2. Active Sets

The states generated at each stage by expression (9) do not include all the possible subsets of N_i but only a subset of them, which we call *active sets*.

We can generate the active set for each stage in advance by simply running the recursion, without computing the values f, but using the transition map defined previously, before starting the full recursion. This approach allows us to apply a modified version of the so-called *threads compaction*, described in [17] and analyzed in [34].

This method consists in creating a mask for allowing the GPU to spawn only the threads useful for the computation. Using this technique and exploiting the property described before, we can take in consideration only the states useful for the relaxation and calibrate the computational resources on these, avoiding the overhead induced by not working threads.

For each stage (q, i) the cardinality of the corresponding active set is given by the function $DimActiveSet(q, i)$ and the index of the k-th set belonging to the active set is given by function $ActiveSet(q, i, k)$.

4.3.3. Enhanced Serial Version

The use of active sets at each stage, together with the indexing for the sets NG, improve also the performance of the serial version of the method, as shown in the following.

ALGORITHM NG-PATHS ENHANCED$(n, Q, \mathbf{d}, \mathbf{q})$

```
1  // Initialization
2  for q = 0, ..., Q do
3      for i = 1, ..., n do
4          if q_i(i) = q
5              then f(q, i, 0) = d(0, i), π(q, i, 0) = 0
6              else  f(q, i, 0) = ∞, π(q, i, 0) = −1
7  // ng-Paths
8  for q = 0, ..., Q do
```

ACCEPTED MANUSCRIPT

```
9     for i = 1, ..., n do
10        for j = 1, ..., n do
11           for h = 0, ..., DimActiveSet(q − q_i(i), j)| do
12              NG_index = ActiveSet(q − q_i, j, h)
13              NG'_index = Map(i, j, NG_index)
14              if f(q, i, NG'_index) > f(q − q_i(i), j, NG_index) + d(i, j)
15                 then f(q, i, NG'_index) = f(q − q_i(i), j, NG_index) + d(i, j)
16                      π(q, i, NG'_index) = j
17 return f, π
```

In this improved version of the serial algorithm NG-Paths, functions **f** and $\boldsymbol{\pi}$ make use of an index, instead of using the whole set NG. The functions *ActiveSet* and *Map* at lines 12 and 13 are nothing more than a direct access to data structures saved on two arrays.

4.3.4. Parallel GPU Version

The parallel version for GPU of the algorithm NG-Paths Enhanced is given in the following.

GPU-NG-Paths($n, Q, \mathbf{d}, \mathbf{q}, \mathbf{ActiveSet}, \mathbf{Map}$)

```
1  Variables f and π are initialized as in the serial algorithm
2  // Number of Active Threads for each set of stages q
3  for q = 0, ..., Q do
4     for i = 1, ..., n do
5        ActThds(q) = ActThds(q) + DimActiveSets(q, i)
6  // Initialize StartLabels
7  for q = 0, ..., Q do
8     for i = 1, ..., n do
9        for h = 0, ..., DimActiveSets(q, i) do
10          NG_index = ActiveSet(q, i, h)
11          SLabels_N(q).Add(i)
```

ACCEPTED MANUSCRIPT

12 $SLabels_{NG}(q)$.Add(NG_{index})
13 // Kernel Setup
14 BLOCKS $B = (ActThds(q)/T, n-1)$, THREADS T
15 // Main Loop
16 **for** $q' = 0, \ldots, Q$ **do**
17 NG-PATHS-KERNEL<<< $BLKS, THDS$ >>>
18 (q', **q**, **d**, **f**, $\boldsymbol{\pi}$, **ActiveSet**, **Map**, $\mathbf{SLabels_N}$, $SLabels_{NG}$)
19 **return** $\boldsymbol{f}, \boldsymbol{\pi}$

Inside the main procedure, GPU-NG-PATHS, at lines 5-7, we count the number of active labels for each stage q. Using this value, inside the main loop of the procedure, we spawn the necessary number of threads for each iteration of the loop (line 19). At lines 9-14 we initialize the $StartLabels_N$ and $StartLabels_{NG}$ structures, containing, for each q, the indices of the active NG for each vertex. As for the serial version of the algorithm, we return the array f with the function values with the array of the predecessor vertex, π_n and the array of the predecessor path π_{ng} (line 23).

NG-PATHS-KERNEL(q', **q**, **d**, **f**, $\boldsymbol{\pi}$, **ActiveSet**, **Map**, $\mathbf{SLabels_N}$, $\mathbf{SLabels_{NG}}$)
1 $idx = blockIdx.x * blockDim.x + threadIdx.x$
2 $i = blockIdx.y$, $j = SLabels_N(q', idx)$, $NG_{index} = SLabels_{NG}(q', idx)$
3 $NG'_{index} = Map(i, j, NG_{index})$
4 **if** $f(q' + q_i, i, NG'_{index}) > f(q', j, NG_{index}) + d(i, j)$
5 **then** $f(q' + q_i, i, NG'_{index}) = f(q', j, NG_{index}) + d(i, j)$
6 $\pi(q' + q_i, i, NG'_{index}) = j$

Inside the GPU kernel, NG-PATHS-KERNEL, at lines 1-3, we define the indices (i, j, NG_{index}) of the label. To retrieve the j and NG_{index} we use the idx index for each thread that enumerates a specific location of the $SLabels_N$ and $SLabels_{NG}$ lists. In this case, as shown at line 14 of the main algorithm *GPU-NG-PATHS*, we use bi-dimensional blocks: the x dimension for managing

ACCEPTED MANUSCRIPT

the indices of $SLabels_N$ and $SLabels_{NG}$ and the y dimension to enumerate the nodes. At line 4, using the transition map, we find the index if the new NG set for the expanded label, and at lines 5-7 we update, if necessary, the new label. The operation in these lines is implemented using the *AtomicMin()* CUDA primitive to manage the concurrent update of a single variable by more threads simultaneously, in order to avoid race conditions and inconsistent results.

4.4. GPU Algorithms for the Asymmetric Case

In real-world VRPs the cost matrix is usually asymmetric (i.e., $d_{ij} \neq d_{ji}$ for some $i, j \in V$). In this situation, when we also need to compute the *backward* q-paths or ng-paths from a vertex i to the depot, we can use the forward recursions replacing the cost matrix $\mathbf{D}$ with its transposed $\mathbf{D}^T$. Moreover, when we need to compute both forward and backward recursions (e.g., in the through-q-route relaxation), we may further improve the performance by exploiting all the parallel features of the GPU environment. In particular, we may compute concurrently the two recursions on the same GPU, introducing another level of parallelism among kernels.

In our case we do not use streams to hide the memory transactions between the CPU and the GPU, but we execute the same kernel with different data on the same GPU, in a typical SIMD approach. The kernels are essentially those described in the previous paragraph. The main difference is the use of the *cudaMemcpyAsync()* primitive that is a page-locked memory, also known as *pinned memory*.

5. Computational Results

In this section we report the experimental results, where we compare the serial and the GPU parallel versions of the algorithms described in this paper.

The computational experiments are performed on a workstation equipped with an Intel i7 4820K @3.9 GHz with 32 Gigabytes of RAM and a NVIDIA GTX TITAN with 2688 CUDA Cores @837 MHz with 6 Gigabytes of GDDR5 RAM.

ACCEPTED MANUSCRIPT

The computational experiments are carried out on symmetric and asymmetric well-known instances from the literature. In particular, we use some symmetric instances of the CVRPLIB ([35]); twelve large instances proposed by [36]; nine instances from series A, B, and P proposed by [37]; twelve instances from series E, F, and M proposed by [38, 39, 40], respectively. The asymmetric dataset contains eight instances of the VRPLIB ([41]) and two new instances of large size, called Balman859-1000 and Balman859-2000, corresponding to real-word VRPs (downlodable from: `http://astarte.csr.unibo.it/data/#ACVRP`).

In our computational tests, we compare the efficiency of the implementations of the route relaxation algorithms using a serial CPU approach and using GPU computing. The serial and GPU algorithms are functionally equivalent, the only differences concern the implementation on different architectures, as described in Section 4. The implementations of serial and GPU versions are straight and without any peculiar enhancement not described in this paper, however for the serial version we obtained a performance which is not very far from the best performing route relaxations proposed in the literature. In Tables 1–4 we present the results obtained by executing only one relaxed route generation using the original costs and in Table 5 the results obtained by a column generation procedure.

In implementing the GPU algorithms we solved two main issues concerning the choice of the number of threads per block and the retrieval of each state for the NG-path procedure. The number of threads per block was set to 128 threads with the support of the NVIDIA Visual Profiler. Whereas, we performed the retrieval of each state using a mapping computed in a preprocessing.

Tables 1–4 report for each instance its *Name*, the number of vertices n, the capacity Q, and for each algorithm the execution time for a single run. The computing times of the GPU versions also include the time spent for exchanging the data between the CPU and the GPU. We choose to include these times because the relaxations are usually repeatedly called into a column generation algorithm and the time for loading and storing has a significant impact on the performance. Tables 1–4 also report the *speed-up* factor between the serial

ACCEPTED MANUSCRIPT

and the parallel version of each relaxation. The speed-up factor is given by the ratio between the running times of the serial and the parallel algorithm: $SpeedUp = Time_{Serial}/Time_{Parallel}$.

We report the results for the symmetric instances in Tables 1 and 3 and the results with the asymmetric instances in Tables 2 and 4. The instances included in each table are the most time consuming instances that the corresponding algorithms were able to solve. In our computational experiment, the instances used for testing the q-route and the through-q-route relaxations are different from the instances used for testing the ng-route relaxation. The instances used for testing the q-route and the through-q-route relaxations are too large for being solved by the ng-route relaxation, because we did not have enough memory. On the other hand, the instances used for testing the ng-route relaxation are too easy for the q-route and the through-q-route relaxations and the computing times are too small for an effective comparison.

ACCEPTED MANUSCRIPT

ACCEPTED MANUSCRIPT

Table 1: Speed-ups for the symmetric q-path and through-q-route relaxations, instances proposed by [36]

Instances			CPU Computing Times			GPU Computing Times			Speed-Up		
Name	n	Q	q-path	t-q-route	q + t-q-route	q-path	t-q-route	q + t-q-route	q-paths	t-q-route	q + t-q-route
Li_21	560	1200	1.045	2.325	3.370	0.140	0.210	0.350	7.455	11.055	9.615
Li_22	600	900	0.874	1.311	2.185	0.101	0.128	0.228	8.692	10.257	9.568
Li_23	640	1400	1.576	4.695	6.271	0.177	0.284	0.461	8.897	16.527	13.596
Li_24	720	1500	2.215	6.677	8.892	0.251	0.363	0.614	8.831	18.369	14.475
Li_25	760	900	1.857	2.340	4.197	0.162	0.149	0.311	11.483	15.717	13.513
Li_26	800	1700	4.805	11.497	16.302	0.362	0.511	0.873	5.126	22.489	18.663
Li_27	840	900	3.027	2.932	5.959	0.209	0.165	0.373	14.514	17.807	15.967
Li_28	880	1800	7.004	15.241	22.245	0.486	0.627	1.113	14.424	24.289	19.985
Li_29	960	2000	10.046	23.353	33.399	0.628	0.837	1.465	15.995	27.910	22.802
Li_30	1040	2100	13.166	29.640	42.806	0.869	0.994	1.862	15.158	29.831	22.987
Li_31	1120	2300	16.832	40.154	56.986	1.061	1.276	2.336	15.869	31.479	24.392
Li_32	1200	2500	21.092	52.026	73.118	1.328	1.607	2.935	15.878	32.385	24.914
Average			6.962	16.016	22.978	0.481	0.596	1.077	11.860	21.510	17.540

ACCEPTED MANUSCRIPT

ACCEPTED MANUSCRIPT

Table 2: Speed-ups for the asymmetric q-path and through-q-route relaxations, instances of the VRPLIB

Instances			CPU Computing Times			GPU Computing Times			Speed-Up		
Name	n	Q	q-path	t-q-route	q + t-q-route	q-path	t-q-route	q + t-q-route	q-paths	t-q-route	q + t-q-route
A034-02f	34	1000	0.008	0.031	0.039	0.024	0.007	0.031	0.334	4.722	1.278
A036-03f	36	1000	0.015	0.031	0.031	0.024	0.007	0.031	0.619	4.557	0.999
A039-03f	39	1000	0.015	0.031	0.046	0.026	0.007	0.033	0.579	4.213	1.382
A045-03f	45	1000	0.015	0.047	0.062	0.026	0.009	0.034	0.578	5.505	1.799
A048-03f	48	1000	0.016	0.047	0.063	0.025	0.009	0.035	0.634	4.986	1.817
A056-03f	56	1000	0.015	0.063	0.078	0.026	0.011	0.037	0.584	5.777	2.131
A065-03f	65	1000	0.016	0.093	0.109	0.026	0.013	0.039	0.606	7.388	2.796
A071-03f	71	1000	0.031	0.078	0.109	0.026	0.014	0.040	1.184	5.642	2.724
Balman859-1000	859	1000	6.100	3.026	9.126	0.393	0.122	0.515	15.514	24.781	17.710
Balman859-2000	859	2000	13.385	17.113	30.498	0.791	0.502	1.292	16.929	34.102	23.597
Average			1.962	2.056	4.016	0.139	0.070	0.209	3.756	10.167	5.623

ACCEPTED MANUSCRIPT

Table 3: Speed-ups for the symmetric ng-path relaxation, instances of series A, B, E, F, M, and P

Instances			CPU Computing Times					GPU Computing Times					Speed-Up				
Name	n	Q	NG:8	NG:10	NG:12	NG:13	NG:14	NG:8	NG:10	NG:12	NG:13	NG:14	NG:8	NG:10	NG:12	NG:13	NG:14
A-n62-k8	62	100	0.047	0.093	0.281	0.421	0.717	0.004	0.007	0.021	0.047	0.080	12.796	12.510	13.646	8.875	8.947
A-n63-k10	63	100	0.047	0.094	0.234	0.359	0.546	0.003	0.007	0.020	0.046	0.070	14.030	12.974	11.958	7.775	7.813
A-n64-k9	64	100	0.047	0.093	0.249	0.375	0.515	0.003	0.007	0.021	0.034	0.066	14.766	13.358	11.906	10.969	7.830
A-n80-k10	80	100	0.078	0.187	0.484	0.733	1.061	0.005	0.012	0.036	0.067	0.113	14.888	15.004	13.425	10.927	9.356
B-n50-k8	50	100	0.031	0.094	0.296	0.484	0.827	0.003	0.007	0.024	0.042	0.095	10.431	12.835	12.530	11.433	8.715
B-n68-k9	68	100	0.094	0.234	0.639	0.842	1.373	0.006	0.021	0.068	0.088	0.167	14.925	10.928	9.441	9.530	8.220
B-n78-k10	78	100	0.110	0.343	0.998	1.482	2.511	0.010	0.032	0.109	0.199	0.290	11.542	10.754	9.192	7.442	8.659
E-n51-k5	51	160	0.063	0.125	0.359	0.593	0.920	0.004	0.008	0.025	0.062	0.084	16.475	15.543	14.089	9.607	10.916
E-n76-k7	76	220	0.156	0.421	1.279	2.043	3.260	0.010	0.026	0.081	0.194	0.305	15.388	16.476	15.775	10.533	10.680
E-n76-k8	76	180	0.125	0.296	0.921	1.420	2.215	0.008	0.019	0.071	0.139	0.222	16.346	15.811	12.894	10.228	9.975
E-n76-k10	76	140	0.078	0.187	0.530	0.811	1.216	0.005	0.012	0.036	0.067	0.124	14.684	15.352	14.536	12.174	9.773
E-n76-k14	76	100	0.031	0.078	0.203	0.297	0.390	0.003	0.006	0.018	0.044	0.065	10.375	12.597	11.361	6.688	5.998
E-n101-k8	101	200	0.272	0.795	2.371	3.619	5.523	0.017	0.046	0.182	0.306	0.528	15.826	17.271	13.042	11.824	10.453
E-n101-k14	101	112	0.110	0.280	0.765	1.108	1.623	0.007	0.018	0.061	0.098	0.170	15.621	15.862	12.535	11.344	9.520
F-n135-k7	135	2210	9.516	30.342	91.853	–	–	0.480	1.634	5.694	–	–	19.811	18.565	16.131	–	–
M-n121-k7	121	200	0.671	2.169	9.345	19.407	38.876	0.045	0.246	1.203	2.710	5.731	14.871	8.817	7.767	7.160	6.783
M-n151-k12	151	200	0.577	1.591	4.181	6.692	10.358	0.034	0.099	0.323	0.556	0.978	16.731	16.083	12.952	12.038	10.587
M-n200-k16	200	200	0.983	2.901	7.301	10.905	15.928	0.064	0.198	0.611	0.956	1.524	15.292	14.629	11.946	11.408	10.450
M-n200-k17	200	200	0.998	2.902	7.332	10.888	15.943	0.064	0.197	0.617	0.966	1.527	15.501	14.759	11.887	11.272	10.443
P-n50-k8	50	120	0.031	0.047	0.109	0.171	0.250	0.002	0.005	0.012	0.024	0.044	14.299	9.918	8.804	7.001	5.685
P-n70-k10	70	135	0.047	0.125	0.359	0.530	0.749	0.004	0.009	0.035	0.048	0.089	12.401	13.832	10.362	10.954	8.439
Average			0.672	2.067	6.195	3.159	5.240	0.037	0.125	0.441	0.335	0.614	14.619	13.994	12.199	9.959	8.962

ACCEPTED MANUSCRIPT

ACCEPTED MANUSCRIPT

ACCEPTED MANUSCRIPT

Table 4: Speed-ups for the asymmetric ng-path relaxation, instances of the VRPLIB

Instances			CPU Computing Times				GPU Computing Times				Speed-Up			
Name	n	Q	NG:8	NG:10	NG:12	NG:13	NG:14	NG:8	NG:10	NG:12	NG:13	NG:14	NG:8	NG:10	NG:12	NG:13	NG:14
A034-02f	34	1000	0.422	1.045	3.151	5.055	8.096	0.022	0.035	0.086	0.128	0.199	19.055	29.825	36.821	39.555	40.595
A036-03f	36	1000	0.406	1.014	3.230	4.711	7.629	0.023	0.037	0.086	0.127	0.194	17.849	27.397	37.417	37.189	39.292
A039-03f	39	1000	0.406	1.185	3.417	5.289	8.205	0.024	0.038	0.091	0.146	0.221	17.256	31.399	37.441	36.216	37.103
A045-03f	45	1000	0.546	1.404	4.259	6.942	12.137	0.027	0.053	0.138	0.238	0.443	19.884	26.447	30.809	29.206	27.377
A048-03f	48	1000	0.671	1.809	5.835	9.625	18.205	0.030	0.079	0.239	0.427	0.956	22.587	22.900	24.374	22.565	19.041
A056-03f	56	1000	0.905	2.730	7.784	14.367	21.918	0.041	0.114	0.331	0.687	1.102	22.124	23.856	23.514	20.910	19.898
A065-03f	65	1000	1.154	3.213	9.423	15.398	25.365	0.053	0.122	0.398	0.720	1.298	21.691	26.385	23.661	21.391	19.547
A071-03f	71	1000	1.326	3.869	11.684	19.625	30.639	0.057	0.160	0.572	1.109	1.813	23.425	24.127	20.428	17.703	16.904
Average			0.730	2.034	6.098	10.127	16.524	0.035	0.080	0.243	0.448	0.778	20.484	26.542	29.308	28.092	27.470

ACCEPTED MANUSCRIPT

ACCEPTED MANUSCRIPT

Table 1 shows that for the symmetric instances the speed-ups for the q-path range from a minimum of 5.1 up to 15.9, whereas for the through-q-route from 10.2 to 32.3. Therefore, the through-q-route is able to make better use of the GPU than the q-path relaxation. Since, the through-q-route needs the computation of the q-path, Table 1 also reports in column "q + t-q-route" the overall computing time of the two algorithms. As shown in Table 2, the performance of the GPU version of the q-path for the asymmetric VRPLIB instances shows a performance worse than that the serial version, whereas the performance of the GPU version is much better for instances *Balman859-1000* and *Balman859-2000*. The poor performance increase noticed for the VRPLIB instances is due to the small number of vertices n of these instances.

The ng-route relaxation was tested for different ng-set sizes Δ_i ranging from 8 to 14. For instance *F-n135-k7* when Δ_i is equal to 13 and 14 the memory was not enough for the GPU algorithm and in Table 3 we report the symbol "–". For the symmetric instances the speed-up is on average quite good and a little better for the tests where Δ_i is smaller. Notice that the best performance is obtained for the instance *F-n135-k7*, which has a large Q. Table 4 shows that the speed-up factor is excellent for the asymmetric instances, reaching values up to 40. All these instances are characterized by a large Q, indicating that the GPU version of the ng-route relaxation performs particularly well for the instances where Q is large.

In Tables 3 and 4 we show the computing times for executing a single run of the NG-path procedure. In Table 5 we show the results obtained by a column generation procedure applied to the symmetric instances, when we generate columns by means of the NG-path procedure, setting the NG size equal to 8. For each instance we report the integer solution value z_{Opt}, the LP-relaxation optimal solution value z_{LP}, and we compare the serial CPU version with the OpenMP and the GPU implementations. We also report the computing time T_{NG} required by the NG-path procedure and relate it to the total computing time T_{Tot} spent by the whole column generation procedure.

In our column generation algorithm, in addition to the NG-path procedure,

ACCEPTED MANUSCRIPT

Table 5: Column generation for the symmetric ng-path relaxation with NG size equal to 8, instances of series A, B, E, F, M, and P

Instances			CPU		OpenMP		GPU	
Name	z_{Opt}	z_{LP}	T_{NG}	T_{Tot}	T_{NG}	T_{Tot}	T_{NG}	T_{Tot}
A-n63-k10	1314.000	1283.963	3.959	5.806	1.126	3.326	0.572	3.209
A-n64-k9	1401.000	1364.662	5.392	7.875	1.487	4.470	1.037	4.433
A-n80-k10	1763.000	1729.230	12.119	16.692	3.225	9.267	1.412	6.834
B-n50-k8	1312.000	1266.148	2.010	3.075	0.564	1.844	0.490	2.129
B-n68-k9	1272.000	1198.404	5.479	7.801	1.454	4.468	0.907	3.969
B-n78-k10	1221.000	1163.440	7.954	11.094	2.204	6.173	0.784	4.881
E-n51-k5	521.000	517.135	4.643	6.343	1.280	3.297	0.677	3.157
E-n76-k7	682.000	664.178	23.615	28.833	6.495	13.046	2.008	8.954
E-n76-k8	735.000	717.785	15.459	19.367	4.471	9.290	1.429	6.240
E-n76-k10	830.000	811.766	7.844	10.438	2.298	5.375	1.002	4.294
E-n76-k14	1021.000	1000.192	3.859	5.482	1.126	3.188	0.421	2.625
E-n101-k8	815.000	789.386	59.865	72.529	16.230	31.973	4.062	19.231
E-n101-k14	1067.000	1047.160	17.925	23.425	4.857	12.437	1.536	8.220
F-n135-k7	1162.000	1133.793	6729.317	7167.464	1856.016	2318.125	252.990	630.531
M-n121-k7	1034.000	1025.951	103.759	122.682	27.192	51.624	5.694	23.406
M-n151-k12	1015.000	994.805	150.262	178.480	40.552	74.089	8.177	39.725
M-n200-k16	1274.000	1249.216	252.291	296.508	67.849	117.308	11.661	58.169
M-n200-k17	1275.000	1251.423	260.132	305.927	69.894	121.230	12.600	65.397
P-n50-k8	631.000	614.571	1.670	2.434	0.507	1.485	0.188	1.395
P-n70-k10	827.000	809.278	5.954	7.853	1.672	4.206	0.531	2.825
Average			383.675	415.005	105.525	139.811	15.409	44.981

we solved the master problem by the LP solver of IBM Ilog Cplex 12.6.1 using the default setting, therefore multi-threading was enabled. The remaining code for the column generation was strictly serial. These experiments are performed on a workstation equipped with an Intel Core i7-4790 @3.60 GHz with 32 Gigabytes of RAM and a NVIDIA GTX 980 Ti with 6 Gigabytes of GDDR5 VRAM with 2816 CUDA Cores @1.0 GHz.

Table 5 shows that in the serial CPU implementation the computing time of the NG-path procedure clearly dominates the remaining column generation

ACCEPTED MANUSCRIPT

code. On average, more than the 90% of the computing time is spent by the NG-path procedure. Using OpenMP, the percentage decreases, and with GPU computing the NG-path procedure on average requires only about one third of the total computing time. Notice that the overall computing time was reduced by about nine times just implementing the NG-path procedure using GPU computing.

6. Conclusions

In this paper we use GPU computing for implementing effective q-route and ng-route relaxations. We studied tailored data structures and fine-tuned procedures in order to achieve the maximum efficiency, both for the serial and for the parallel versions of the dynamic programming recursions at the heart of the relaxations.

The computational results prove the effectiveness of the GPU version of the relaxations. On average, the speed-up is sufficiently large to be extremely competitive with the serial implementation and suggest that parallel codes could be an important building block of state of the art exact methods, even for industrial combinatorial optimization problems.

References

[1] G. Dantzig, J. Ramser, The truck dispatching problem, Management Science 6 (1) (1959) 80–91.

[2] P. Toth, D. Vigo (Eds.), The Vehicle Routing Problem, Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 2001.

[3] B. Golden, S. Raghavan, E. Wasil (Eds.), The Vehicle Routing Problem: Latest Advances and New Challenges, Vol. 43 of Operations Research/Computer Science Interfaces Series, Springer, 2008.

[4] http://www.verolog.eu/ (2014).

ACCEPTED MANUSCRIPT

[5] M. Boschetti, V. Maniezzo, A set covering based matheuristic for a real-world city logistics problem, International Transactions in Operational Research 22 (1) (2015) 169–195.

[6] M. Boschetti, V. Maniezzo, M. Roffilli, A. Bolufe Rohler, Matheuristics: Optimization, simulation and control, in: M. Blesa, C. Blum, L. Di Gaspero, A. Roli, M. Samples, S. A. (Eds.), Hybrid Metaheuristics, Vol. 5818 of LNCS, Springer, 2009, pp. 171–177.

[7] W. Ou, B.-G. Sun, A dynamic programming algorithm for vehicle routing problems, in: Proceedings of the 2010 International Conference on Computational and Information Sciences, ICCIS '10, IEEE Computer Society, Washington, DC, USA, 2010, pp. 733–736.

[8] N. Christofides, A. Mingozzi, P. Toth, Exact algorithms for the vehicle routing problem based on spanning tree and shortest path relaxation, Mathematical Programming 10 (1981) 255–280.

[9] J. Lysgaard, A. Letchford, R. Eglese, A new branch-and-cut algorithm for the capacitated vehicle routing problem, Mathematical Programming 100 (2) (2004) 423–445.

[10] M. Desrochers, J. Desrosiers, M. Solomon, A new optimization algorithm for the vehicle routing problem with time windows, Operations Research 40 (2) (1992) 342–354.

[11] R. Baldacci, A. Mingozzi, R. Roberti, New route relaxation and pricing strategies for the vehicle routing problem, Operations Research 59 (2011) 1269–1283.

[12] R. Martinelli, D. Pecin, M. Poggi, Efficient elementary and restricted non-elementary route pricing, European Journal of Operational Research 239 (1) (2014) 102–111.

ACCEPTED MANUSCRIPT

[13] A. Brodtkorb, T. Hagen, C. Schulz, G. Hasle, GPU computing in discrete optimization. Part I: introduction to the GPU, EURO Journal on Transportation and Logistics 2 (1-2) (2013) 129–157.

[14] C. Schulz, G. Hasle, A. R. Brodtkorb, T. Hagen, GPU computing in discrete optimization. Part II: survey focused on routing problems, EURO Journal on Transportation and Logistics 2 (1-2) (2013) 159–186.

[15] J. E. Stone, D. J. Hardy, I. S. Ufimtsev, K. Schulten, GPU-accelerated molecular modeling coming of age, Journal of Molecular Graphics and Modelling 29 (2) (2010) 116–125.

[16] V. Boyer, D. El Baz, E. Moussa, Solving knapsack problems on GPU, Computers & Operations Research 39 (1) (2012) 42–47.

[17] P. Harish, V. Vineet, P. Narayanan, Large graph algorithms for massively multithreaded architectures, Tech. rep., Centre for Visual Information Technology, Institute of Information Technology, Hyderabad, India (2009).

[18] A. Buluç, J. Gilbert, C. Budak, Solving path problems on the GPU, Parallel Computing 36 (5) (2010) 241–253.

[19] H. Ortega-Arranz, Y. Torres, D. Llanos, A. Gonzalez-Escribano, A new GPU-based approach to the shortest path problem, in: High performance computing and simulation (HPCS), International Conference, IEEE, 2013, pp. 505–511.

[20] S. Kumar, A. Misra, R. Tomar, A modified parallel approach to single source shortest path problem for massively dense graphs using CUDA, in: Computer and Communication Technology (ICCCT), 2nd International Conference, IEEE, 2011, pp. 635–639.

[21] G. Katz, J. Kider, Jr, All-pairs shortest-paths for large graphs on the GPU, in: Proceedings of the 23rd ACM SIGGRAPH/EUROGRAPHICS sympo-

ACCEPTED MANUSCRIPT

sium on Graphics Hardware, GH '08, Eurographics Association, Aire-la-Ville, Switzerland, Switzerland, 2008, pp. 47–55.
URL http://dl.acm.org/citation.cfm?id=1413957.1413966

[22] B. Lund, J. Smith, A multi-stage CUDA kernel for Floyd-Warshall, CoRR abs/1001.4108.
URL http://arxiv.org/abs/1001.4108

[23] M. Boschetti, V. Maniezzo, F. Strappaveccia, Using GPU computing for solving the two-dimensional guillotine cutting problem, INFORMS Journal on Computing 28 (3) (2016) 540–552.

[24] M. Balinski, R. Quandt, On an integer program for a delivery problem, Operations Research 12 (2) (1964) 300–304.

[25] M. Dror, Note on the complexity of the shortest path models for column generation in VRPTW, Operations Research 42 (1994) 977–978.

[26] D. Feillet, P. Dejax, M. Gendreau, C. Gueguen, An exact algorithm for the elementary shortest path problem with resource constraints: Application to some vehicle routing problems, Networks 44 (2004) 216–229.

[27] G. Righini, M. Salani, New dynamic programming algorithms for the resource constrained elementary shortest path, Networks 51 (3) (2008) 155–170.

[28] N. Christofides, A. Mingozzi, P. Toth, State-space relaxation procedures for the computation of bounds to routing problems, Networks 11(2) (1981) 145–164.

[29] G. Righini, M. Salani, Decremental state space relaxation strategies and initialization heuristics for solving the orienteering problem with time windows with dynamic programming, Computers & Operations Research 36 (4) (2009) 1191–1203.

ACCEPTED MANUSCRIPT

[30] G. Righini, M. Salani, Symmetry helps: bounded bi-directional dynamic programming for the elementary shortest path problem with resource constraints, Discrete Optimization 3 (3) (2006) 255–273.

[31] S. Irnich, D. Villeneuve, The shortest-path problem with resource constraints and k-cycle elimination for $k \geq 3$, INFORMS Journal on Computing 18 (3) (2006) 391–406.

[32] Nvidia, Nvidia developer zone, https://developer.nvidia.com/, Accessed: 2013-12-03 (2007).

[33] M. Harris, Optimizing parallel reduction in CUDA, Nvidia developer zone. URL `http://developer.download.nvidia.com/assets/cuda/files/reduction.pdf`

[34] C. Schulz, Efficient local search on the GPU – investigations on the vehicle routing problem, Journal of Parallel and Distributed Computing 73 (1) (2013) 14–31.

[35] CVRPLIB, Capacitated vehicle routing problem library, http://vrp.atd-lab.inf.puc-rio.br/index.php/en/, Accessed: 2015-09-18 (2015).

[36] F. Li, B. Golden, E. Wasil, Very large-scale vehicle routing: new test problems, algorithms, and results, Computers & Operations Research 32 (5) (2005) 1165–1179.

[37] P. Augerat, J. M. Belenguer, E. Benavent, A. Corberán, D. Naddef, G. Rinaldi, Computational results with a branch and cut code for the capacitated vehicle routing problem, Tech. Rep. 949-M, ARTEMIS-IMAG, Université Joseph Fourier, Grenoble, France (1995).

[38] N. Christofides, S. Eilon, An algorithm for the vehicle-dispatching problem, Operational Research Quarterly 20 (3) (1969) 309–318.

[39] M. Fisher, Optimal solution of vehicle routing problem using minimum k-trees, Operations Research 42 (1994) 626–642.

ACCEPTED MANUSCRIPT

[40] N. Christofides, A. Mingozzi, P. Toth, Exact algorithms for the vehicle routing problem, based on spanning tree and shortest path relaxations, Mathematical Programming 20 (1981) 255–282.

[41] VRPLIB, Vehicle Routing Problem LIBrary, http://or.dei.unibo.it/library/vrplib-vehicle-routing-problem-library, Accessed: 2015-09-18 (2015).

ACCEPTED MANUSCRIPT